OPERATING SYSTEMS LAB PROJECT REPORT

Page Replacement Algorithm Simulator Using Python

Submitted by: Shalini Sharma

Course: B.Tech – Computer Science Engineering (Data Analytics)

Subject: Operating Systems Lab

Project Title: Page Replacement Algorithm Simulator

Programming Language: Python

Submission Date: 25 October 2026

1. Introduction

An operating system manages the main memory of a computer. During program execution, pages are loaded into memory frames. When all frames are occupied and a new page is required, the operating system must select an existing page for replacement.

A page replacement algorithm determines which page should be removed from memory.

This project implements three page replacement algorithms: FIFO (First In, First Out), LRU (Least Recently Used), and Optimal Page Replacement. The simulator accepts a page reference string and frame capacity from the user. It displays the execution steps and calculates page hits, page faults, hit ratio, and fault ratio.

2. Objectives

1. To understand page replacement in operating systems.
2. To implement FIFO, LRU, and Optimal algorithms using Python.
3. To accept user input for page references and frame capacity.

4. To display step-by-step frame contents.
5. To calculate page hits, page faults, hit ratio, and fault ratio.
6. To compare the performance of the three algorithms.

3. Software Requirements

- Programming language: Python 3
- Editor: Visual Studio Code or any Python-compatible editor
- Operating system: Windows, Linux, or macOS
- External libraries: None

4. Algorithm Descriptions

4.1 FIFO (First In, First Out)

FIFO replaces the page that entered memory first. A pointer tracks the next frame to replace when all frames are full.

Advantages: Simple to understand and implement.

Limitation: It does not consider how frequently or recently a page is used. FIFO can also exhibit Belady's anomaly, where increasing the number of frames can sometimes increase page faults.

4.2 LRU (Least Recently Used)

LRU replaces the page that has not been used for the longest time. It tracks the most recent use of each page.

Advantages: Uses past access history and can perform better than FIFO on some workloads.

Limitation: Tracking recent page usage introduces additional overhead.

4.3 Optimal Page Replacement

Optimal replaces the page whose next use is farthest in the future. If a page will never be used again, it is a candidate for replacement.

Advantages: Produces the minimum possible number of page faults for a given reference string and frame capacity.

Limitation: A real operating system generally cannot know future page references. Therefore, Optimal is primarily used as a theoretical benchmark.

5. Working of the Simulator

1. The user enters a page reference string.
2. The user enters the number of available frames.
3. The program executes FIFO, LRU, and Optimal independently.
4. Each reference is classified as a HIT or FAULT.
5. The current frame contents are displayed after each reference.
6. The program calculates hits, faults, and ratios.
7. A final table compares the algorithms.

6. Performance Formulas

Let:

- N = Total number of page references
- H = Total page hits
- F = Total page faults

Page Hit: The requested page is already in memory.

Page Fault: The requested page is not present in memory.

Hit Ratio = H / N

Fault Ratio = F / N

Percentage = Ratio × 100

The following relationship must hold:

H + F = N

7. Sample Input

Page reference string:

1 2 3 1 4 2 5

Number of frames:

8. Sample Results

Algorithm	Hits	Faults	Hit Ratio	Fault Ratio
FIFO	2	5	28.57%	71.43%
LRU	1	6	14.29%	85.71%
Optimal	2	5	28.57%	71.43%

The detailed step-by-step traces are provided in the manual tracing section.

9. Performance Analysis

For the selected reference string, FIFO and Optimal each produce five page faults, while LRU produces six.

FIFO is easy to implement but ignores usage history. LRU considers past usage, although it does not necessarily produce fewer faults for every input. Optimal considers future usage and provides a theoretical lower bound on page faults.

The results depend on the page reference string and the number of frames. Therefore, the algorithms must be compared using the same input.

10. Testing and Validation

The simulator should be tested with the following cases:

1. Normal reference string with three frames.
2. A reference string containing repeated pages.
3. A reference string shorter than the frame capacity.
4. An empty reference string.
5. Zero or negative frame capacity.
6. Invalid non-integer input.
7. Negative page numbers.

The program rejects empty input, invalid integers, negative page numbers, and non-positive frame capacity.

11. Applications

- Learning operating-system memory management.

- Understanding page hits and page faults.
- Comparing page replacement strategies.
- Practising algorithm tracing and performance analysis.

12. Conclusion

The Page Replacement Algorithm Simulator successfully demonstrates FIFO, LRU, and Optimal page replacement using Python. It accepts user input, displays frame contents at each step, calculates performance metrics, and compares the algorithms.

This project improves understanding of memory management, algorithm implementation, debugging, and performance evaluation.

13. Future Scope

- Add a graphical user interface.
- Display charts comparing page faults.
- Support additional algorithms such as LFU and Second Chance.
- Allow users to save execution reports automatically.

14. References

1. Operating Systems Lab notes.
2. Operating System Concepts by Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
3. Python 3 official documentation: https://docs.python.org/3/